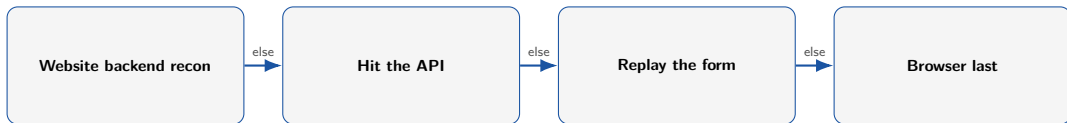


- 1 Be careful with government websites
- 2 Don't scrape what you don't need
- 3 **ALWAYS** check robots.txt

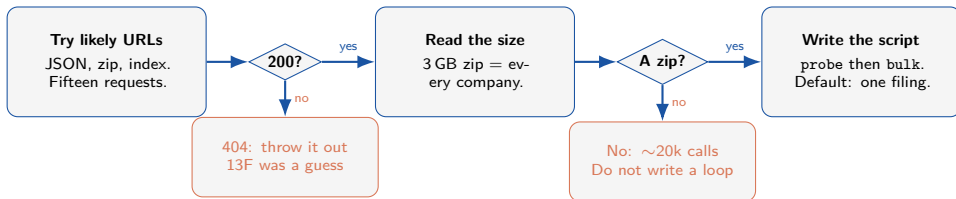
TODAY's example: SEC Edgar filings

Workflow:



Worked example: `fall-2026/sec_edgar.py`.

We did not scrape the search page. We asked what it talks to.



Every request needs a `User-Agent` with your name and email. Without it you get 403, not data.

Fetch the page (or have the agent fetch it) and search the source. The markers below pick the method.

Marker	First move
.zip, .csv, Download	Take the file.
/api/, .json	Call it. Skip the HTML.
MapServer, FeatureServer	ArcGIS. Query the layer; start with a count.
__VIEWSTATE, .aspx	WebForms. Replay the POST. Hidden tokens go stale after each click.
_csrf, JSESSIONID	Session token. Keep the cookie across the run.

Then hit the candidate URLs. Record status, size, row count, whether auth is real, and the pagination parameter (page, from, offset).

Write `scrape/sitename_endpoints.md`. Pick the extract after that file exists.

What you tell the agent

Skips the cheap path

```
Scrape this website and  
give me a CSV.
```

Finds it

```
Probe candidate URLs.  
Report status, size,  
record count, and auth.  
Write endpoints.md.  
Do not write a scraper yet.
```

Once the path is known, from `sec_edgar.py`:

Habit	Why
Safe default	No arguments: one filing under 500 KB. Bulk needs <code>-yes</code> .
Write, then rename	A crash mid-write otherwise leaves a short file that resume treats as done.
Test URLs offline	Builders and parsers have no network. <code>selftest</code> is <code>assert</code> .
One bad record	At 20,000 requests some fail for reasons unrelated to your code. Crashing on item 4,000 is how you lose a night.
Full-text search reports <code>{"value": 10000, "relation": "gte"}</code> . That is a floor. Treating it as the count drops the rest of the sample. The script refuses <code>gte</code> queries and tells you to slice by date.	

Paste this first. Stop when the file exists.

```
Target: https://www.sec.gov/edgar/  
Goal: find the real backend/API behind EDGAR and  
document every bulk data path.  
Deliverable: scrape/sec_edgar_endpoints.md -- host,  
base paths, auth, record counts, pagination params,  
bulk-download sizes, rate limits, with live  
verification evidence.  
Depth: recon only. Do not build a scraper yet.
```